

Padrões de Teste e Qualidade de Código

Eduardo Henrique Moreira dos Santos

Maio 2026

1 Introdução

Este relatório documenta a aplicação de padrões de criação de dados e test doubles em uma suíte de testes para o serviço de checkout de um sistema de e-commerce. O objetivo é demonstrar como o uso deliberado de Builders e Test Doubles — Stubs e Mocks — contribui para testes mais legíveis, manuteníveis e confiáveis.

A seguir, discutimos a escolha do padrão Builder em detrimento do Object Mother, analiso a distinção entre Stubs e Mocks no contexto do teste implementado e concluo com uma reflexão sobre a sustentabilidade da suíte de testes resultante.

O repositório está disponível em <https://github.com/Eduuhms/test-pattern>.

2 Padrões de Criação de Dados (Builders)

2.1 Por que CarrinhoBuilder em vez de CarrinhoMother?

O padrão **Builder** foi escolhido em vez do **Object Mother** por razões práticas de design e escalabilidade. A abordagem Mother cria métodos estáticos para cada combinação de estado necessária, o que leva rapidamente a uma explosão combinatória: cada nova variação exige um novo método, e a classe cresce de forma descontrolada. Além disso, os métodos são pouco flexíveis, se um teste precisa de uma configuração não prevista, é necessário criar um método novo ou adaptar um existente.

O Builder resolve esses problemas com uma interface fluente que permite compor qualquer combinação

a partir de um conjunto fixo de métodos encadeados. O trecho abaixo ilustra a diferença entre as duas abordagens:

Listing 1: Object Mother: métodos estáticos combinatórios

```
// Mother: explosao combinatoria
class CarrinhoMother {
    static umCarrinhoComDoisItens() { ... }
    static umCarrinhoPremiumComTresItens() {
        ... }
    static umCarrinhoVazio() { ... }
    // cresce indefinidamente
}
```

Listing 2: Builder: composição fluente e flexível

```
// Builder: qualquer combinacao
const carrinho = new CarrinhoBuilder()
    .comUser(UserMother.umUsuarioPremium())
    .comItens([new Item('Notebook', 2500)])
    .build();
```

A tabela a seguir resume as diferenças práticas entre os dois padrões:

Aspecto	Mother	Builder
Flexibilidade	Limitada a métodos pre-definidos	Infinitas combinações
Manutenção	Novo atributo = vários métodos	Novo atributo = um método
Legibilidade	Nome do método descreve o estado	Encadeamento descreve o estado
Escalabilidade	Explosão combinatória	Crescimento linear

Table 1: Comparação entre Object Mother e Builder.

2.2 Antes e Depois do Builder

O impacto mais visível do Builder está na redução do volume de setup. No cenário sem o padrão, a preparação do ambiente de teste consumia mais de 25 linhas repetitivas, dificultando a leitura e a identificação do que era realmente relevante para o teste:

Listing 3: Antes: setup manual e verboso

```
const usuario = new User(
  2, 'Maria', 'maria@example.com', 'PREMIUM'
);
const item1 = new Item('Notebook', 1500.00);
const item2 = new Item('Mouse', 50.00);
const item3 = new Item('Teclado', 150.00);
const carrinho = new Carrinho(
  usuario, [item1, item2, item3]
);
// ... mais 15 linhas de stubs e mocks
```

Com o Builder, o mesmo setup se torna conciso e expressivo, comunicando a intenção do teste de forma imediata:

Listing 4: Depois: setup com Builder

```
const carrinho = new CarrinhoBuilder()
  .comUser(UserMother.umUsuarioPremium())
  .comItens([
    new Item('Notebook', 1500.00),
    new Item('Mouse', 50.00),
    new Item('Teclado', 150.00)
  ])
  .build();
```

O setup passou de mais de 25 linhas para 8, sem perda de clareza — pelo contrário, a intenção ficou mais explícita.

3 Padrões de Test Doubles (Mocks vs. Stubs)

3.1 Identificação no Teste de Sucesso Premium

No teste que valida o fluxo de checkout para um usuário Premium, três dependências externas foram substituídas por test doubles. A distinção entre Stub e Mock foi feita com base no que cada dependência precisava verificar: **estado** ou **comportamento**.

Componente	Tipo	Razão
GatewayPagamento	Stub	Testamos o resultado (sucesso/falha)
PedidoRepository	Stub	Precisamos apenas do pedido retornado
EmailService	Mock	Verificamos se foi chamado corretamente

Table 2: Classificação dos test doubles utilizados.

3.2 Por que GatewayPagamento é um Stub?

O comportamento da `CheckoutService` muda inteiramente com base no **estado** retornado pelo gateway: se **success** for **false**, o serviço retorna **null** e interrompe o fluxo; se for **true**, continua o processamento. O número de vezes que o gateway é chamado não importa — o que importa é o valor que ele devolve. Por isso, um Stub é suficiente:

Listing 5: GatewayPagamento como Stub

```
const gatewayStub = {
  cobrar: jest.fn()
    .mockResolvedValue({ success: true });
};

// Verifica o estado resultante
expect(resultado).not.toBeNull();
```

3.3 Por que EmailService é um Mock?

O envio de e-mail é um **efeito colateral crítico** do negócio: não basta que o serviço exista e retorne algo, precisamos garantir que ele foi chamado exatamente uma vez, com o endereço correto, o assunto correto e uma mensagem contendo o número do pedido. Isso é verificação de **comportamento**, que exige um Mock:

Listing 6: EmailService como Mock

```
const emailMock = {
  enviarEmail: jest.fn()
    .mockResolvedValue(undefined)
};

// Verifica o comportamento
expect(emailMock.enviarEmail)
  .toHaveBeenCalledTimes(1);
expect(emailMock.enviarEmail)
  .toHaveBeenCalledWith
```

```
'maria@example.com',
'Seu Pedido foi Aprovado!',
expect.stringContaining('Pedido 123')
);
```

Em síntese: o Gateway importa pelo *resultado* que retorna; o Email importa pela *ação* que executa.

4 Sustentabilidade da Suíte de Testes

4.1 Test Smells Combatidos

A aplicação dos padrões descritos endereça diretamente quatro test smells recorrentes em suítes de testes que crescem sem disciplina:

Setup-Heavy Tests: resolvido pelos Builders, que reduzem blocos de setup de 25 linhas para menos de 10, sem perda de expressividade.

Fragile Tests: resolvido pelos Test Doubles, que isolam o teste de mudanças em dependências externas. Se o gateway trocar de tecnologia, apenas o Stub precisa ser atualizado.

Slow Tests: resolvido pela substituição de chamadas reais a banco de dados, APIs e serviços de e-mail por doubles em memória, mantendo a suíte em milissegundos.

Unclear Expectations: resolvido pelos Mocks, que tornam explícito quais comportamentos são verificados, facilitando a leitura por qualquer desenvolvedor.

4.2 Contribuição para Manutenção a Longo Prazo

A principal contribuição dos padrões está na capacidade de **absorver mudanças sem propagar quebras**. Adicionar um novo atributo ao Carrinho, por exemplo, requer apenas um novo método no Builder; todos os testes existentes continuam funcionando sem alteração. Da mesma forma, novos cenários são criados pela composição de métodos já existentes, sem multiplicar métodos estáticos.

Listing 7: Extensão do Builder sem quebrar testes antigos

```
// Novo atributo: apenas 1 novo metodo
comCupom(cupom) {
  this.cupom = cupom;
  return this;
}

// Novo teste: composicao direta
const carrinho = new CarrinhoBuilder()
  .comUser(UserMother.umUsuarioPremium())
  .comItens([...])
  .comCupom('DESCONTO10')
  .build();
```

5 Conclusão

O uso deliberado de Builders e Test Doubles produziu uma suíte de testes mais legível, manutenível e confiável. O Builder eliminou a redundância do setup e tornou cada teste autoexplicativo. A distinção rigorosa entre Stubs e Mocks deixou claro o que cada teste verifica — estado ou comportamento — reduzindo a ambiguidade e facilitando a evolução do código.

O resultado prático é uma suíte que cresce com o projeto em vez de resistir a ele: novos cenários são baratos de adicionar, mudanças em dependências externas não propagam quebras desnecessárias, e a intenção de cada teste permanece legível independentemente de quem o escreveu.